

Índice

1. Descripción del pipeline ETL	2
2. Tabla de staging: stg_evento_telemetria	2
2.1. Justificación del diseño	2
2.2. DDL	2
2.3. Muestra de TSV crudo	3
3. Tabla de log de errores: log_errores_carga	3
3.1. DDL	3
3.2. Motivos de rechazo	4
4. Carga del TSV al staging	4
5. Transformación staging → core (8 pasos)	4
5.1. Paso 1 — Campos obligatorios nulos o vacíos	5
5.2. Paso 2 — Tipos de datos inválidos	5
5.3. Paso 3 — Valores fuera de rango	5
5.4. Paso 4 — FK inválidas	6
5.5. Paso 5 — Usuarios sin consentimiento	6
5.6. Paso 6 — Duplicados internos del staging	6
5.7. Paso 7 — Duplicados contra el core	7
5.8. Paso 8 — Insertar registros válidos	7
6. Reporte post-carga	8
7. Alineación con requerimientos	9

1. Descripción del pipeline ETL

El pipeline de ingestión sigue tres etapas principales, alineadas con los requerimientos RF11 y RNF06:

Etapa	Tabla	Descripción
1	stg_evento_telemetria	Recibe el TSV crudo sin validación de tipos.
2	log_errores_carga	Registra líneas rechazadas con motivo.
3	evento_telemetria	Destino final con datos validados y tipados.

La razón de usar una tabla intermedia de staging es que ningún dato se pierde antes de ser inspeccionado: si una línea es malformada, queda registrada en el log en lugar de descartarse silenciosamente. La transformación ocurre dentro de un bloque `BEGIN / COMMIT`, garantizando atomicidad: o todos los pasos tienen efecto, o ninguno.

`archivo.tsv` $\xrightarrow{\backslash copy}$ `stg_evento_telemetria` $\xrightarrow{8 \text{ pasos}}$ `evento_telemetria` /
`log_errores_carga`

El pipeline tiene **8 pasos** dentro de la transacción, organizados en tres fases: validación de datos (pasos 1–4), filtros de negocio (paso 5), control de duplicados (pasos 6–7) e inserción final (paso 8).

2. Tabla de staging: stg_evento_telemetria

2.1. Justificación del diseño

Todos los campos se declaran como `TEXT` para aceptar cualquier línea del motor sin rechazar por tipo. Las validaciones se aplican después, en la etapa de transformación. Esto satisface directamente RF11 (validar durante la carga) y RNF06 (trazabilidad de errores).

2.2. DDL

```

1 CREATE TABLE stg_evento_telemetria (
2     raw_id_partida    TEXT,
3     raw_id_jugador    TEXT,
4     raw_tic           TEXT,
5     raw_pos_x         TEXT,
6     raw_pos_y         TEXT,
7     raw_pos_z         TEXT,
8     raw_angulo        TEXT,
9     raw_momentum_x    TEXT,
10    raw_momentum_y    TEXT,
11    raw_momentum_z    TEXT,
12    raw_fov           TEXT,
13    raw_salud         TEXT,
14    raw_armadura      TEXT,

```

```

15 raw_municion      TEXT,
16 raw_id_sector    TEXT,
17 archivo_origen   TEXT      NOT NULL DEFAULT 'sin_origen',
18 cargado_en       TIMESTAMP NOT NULL DEFAULT NOW()
19 );

```

Listing 1: Tabla de staging — todos los campos en TEXT

2.3. Muestra de TSV crudo

El motor emite 15 columnas separadas por tabulador (`\t`), con cabecera en la primera línea. La tabla siguiente muestra las 15 columnas completas:

id_p	id_j	tic	pos_x	pos_y	pos_z	ang	mx	my	mz	fov	sal	arm	mun	sec
1	1	0	1024.0	-256.0	0.0	90.0	2.5	0.0	0.0	90.0	100	0	50	10
1	1	1	1028.5	-254.2	0.0	91.3	2.5	0.0	0.0	90.0	98	0	50	10
1	2	0	512.0	128.0	0.0	45.0	0.0	1.8	0.0	90.0	100	50	30	7
1	2	1	514.8	130.1	0.0	46.0	0.0	1.8	0.0	90.0	100	50	29	7

Abreviaturas: `id_p`=`id_partida`, `id_j`=`id_jugador`, `ang`=ángulo, `mx/my/mz`=momentum `_x/y/z`, `sal`=salud, `arm`=armadura, `mun`=munición, `sec`=`id_sector`.

3. Tabla de log de errores: `log_errores_carga`

3.1. DDL

```

1 CREATE TABLE log_errores_carga (
2   id_log          SERIAL          PRIMARY KEY,
3   origen          VARCHAR(100)   NOT NULL,
4   linea_raw       TEXT           NOT NULL,
5   motivo          VARCHAR(200)   NOT NULL,
6   registrado_en   TIMESTAMP      NOT NULL DEFAULT NOW()
7 );

```

Listing 2: Tabla de log de errores

3.2. Motivos de rechazo

Motivo	Descripción
Campo obligatorio nulo	<code>id_partida</code> , <code>id_jugador</code> o <code>tic</code> vacíos.
Tipo de dato inválido	Campos numéricos con valores no convertibles.
Valor fuera de rango	<code>tic < 0</code> , <code>salud < 0</code> , <code>armadura < 0</code> o <code>municion < 0</code> .
FK: jugador inexistente	<code>id_jugador</code> no existe en la tabla <code>jugador</code> .
FK: partida inexistente	<code>id_partida</code> no existe en la tabla <code>partida</code> .
FK: sector inválido	<code>id_sector</code> no existe o no pertenece al mapa de la partida.
Sin consentimiento	El voluntario tiene <code>consentimiento = FALSE</code> .
Duplicado en staging	Fila repetida dentro del mismo lote TSV; se conserva la primera.
Duplicado contra core	La tripleta (<code>id_partida</code> , <code>id_jugador</code> , <code>tic</code>) ya existe en <code>evento_telemetria</code> .

4. Carga del TSV al staging

```

1  -- Descomentar para recarga de lotes corregidos:
2  -- TRUNCATE stg_evento_telemetria;
3
4  \copy stg_evento_telemetria (
5      raw_id_partida, raw_id_jugador, raw_tic,
6      raw_pos_x, raw_pos_y, raw_pos_z,
7      raw_angulo,
8      raw_momentum_x, raw_momentum_y, raw_momentum_z,
9      raw_fov,
10     raw_salud, raw_armadura, raw_municion,
11     raw_id_sector
12 )
13 FROM '/ruta/al/archivo.tsv'
14 WITH (FORMAT text, DELIMITER E'\t', HEADER true);

```

Listing 3: Comando `\copy` para cargar el TSV

5. Transformación staging → core (8 pasos)

Todo el bloque se ejecuta dentro de `BEGIN / COMMIT`. Cada paso filtra sobre los registros que pasaron todos los anteriores, garantizando que un registro rechazado no avance.

5.1. Paso 1 — Campos obligatorios nulos o vacíos

Se descartan las líneas donde `id_partida`, `id_jugador` o `tic` son nulos o vacíos. Sin estos tres valores no es posible identificar ni clasificar el evento.

```

1 INSERT INTO log_errores_carga (origen, linea_raw, motivo)
2 SELECT archivo_origen,
3        concat_ws(E'\t', raw_id_partida, raw_id_jugador, raw_tic, ...)
4        ,
5        'Campo obligatorio nulo o vacio: id_partida / id_jugador / tic'
6 FROM stg_evento_telemetria
7 WHERE raw_id_partida IS NULL OR TRIM(raw_id_partida) = ''
8        OR raw_id_jugador IS NULL OR TRIM(raw_id_jugador) = ''
9        OR raw_tic IS NULL OR TRIM(raw_tic) = '';

```

Listing 4: Paso 1 — Nulos en campos clave

5.2. Paso 2 — Tipos de datos inválidos

Se usan expresiones regulares para verificar que cada campo sea convertible al tipo esperado. `raw_tic` usa `^-?\d+$` para permitir negativos y clasificarlos en el paso 3 como error de rango, no de tipo, manteniéndolo consistente con lo declarado en la tabla de motivos. Los campos opcionales (`raw_fov`, `raw_angulo`, `raw_id_sector`) solo se validan si vienen con valor.

```

1 raw_id_partida !~ '^\d+$'
2 OR raw_tic !~ '^-?\d+$' -- negativo = tipo ok, rango
3      invalido
4 OR raw_pos_x !~ '^-?\d+(\.\d+)?$'
5 OR (raw_fov IS NOT NULL AND TRIM(raw_fov) <> ''
6     AND raw_fov !~ '^-?\d+(\.\d+)?$') -- opcional
7 OR (raw_id_sector IS NOT NULL AND TRIM(raw_id_sector) <> ''
8     AND raw_id_sector !~ '^\d+$') -- opcional

```

Listing 5: Paso 2 — Validación de tipos con regex

5.3. Paso 3 — Valores fuera de rango

Aplica las restricciones de dominio del diccionario de datos: `tic` ≥ 0 , `salud` ≥ 0 , `armadura` ≥ 0 , `municion` ≥ 0 . Los valores negativos de `tic` llegan aquí (no al paso 2) porque su tipo sí es válido como entero.

```

1 WHERE [tipos validos del paso 2]
2 AND (
3     raw_tic::INT < 0
4     OR raw_salud::INT < 0
5     OR raw_armadura::INT < 0
6     OR raw_municion::INT < 0
7 )

```

Listing 6: Paso 3 — Validación de rangos

5.4. Paso 4 — FK inválidas

Registros con tipos y rangos válidos pero que referencian entidades inexistentes. Sin este paso esas filas desaparecen silenciosamente del pipeline sin dejar trazabilidad (RF11, RNF06).

Se validan tres FK en subpasos independientes:

- **4a** id_jugador no existe en jugador.
- **4b** id_partida no existe en partida.
- **4c** id_sector no existe o no pertenece al mapa de la partida. Se valida con el JOIN completo `partida → mapa → sector`.

```

1 AND NOT EXISTS (
2   SELECT 1
3   FROM partida p
4   JOIN mapa m      ON m.id_mapa    = p.id_mapa
5   JOIN sector sec  ON sec.id_mapa = m.id_mapa
6   WHERE p.id_partida = s.raw_id_partida::INT
7         AND sec.id_sector = s.raw_id_sector::INT
8 )

```

Listing 7: Paso 4c — Sector validado contra mapa/partida

5.5. Paso 5 — Usuarios sin consentimiento

Solo los datos de voluntarios con `consentimiento = TRUE` pueden pasar al core. Este filtro es obligatorio según las consideraciones éticas del proyecto (sección 1.1 del documento de ética).

```

1 JOIN jugador j ON j.id_jugador = s.raw_id_jugador::INT
2 JOIN usuario u ON u.id_usuario = j.id_usuario
3 WHERE [validaciones 1-4 pasadas]
4     AND u.consentimiento = FALSE

```

Listing 8: Paso 5 — Filtro de consentimiento

5.6. Paso 6 — Duplicados internos del staging

Detecta filas repetidas dentro del mismo lote TSV usando `ROW_NUMBER()`. Se conserva la primera por `cargado_en`; las demás se registran en el log. Sin este paso se eliminarían silenciosamente con `DISTINCT ON` sin dejar trazabilidad.

```

1 WITH candidatos AS (
2   SELECT s.*,
3         ROW_NUMBER() OVER (
4           PARTITION BY s.raw_id_partida::INT,
5                       s.raw_id_jugador::INT,
6                       s.raw_tic::INT
7           ORDER BY s.cargado_en ASC
8         ) AS rn
9   FROM stg_evento_telemetria s

```

```

10 JOIN jugador j ON j.id_jugador = s.raw_id_jugador::INT
11 JOIN usuario u ON u.id_usuario = j.id_usuario
12 WHERE [todas las validaciones 1-5 pasadas]
13 )
14 INSERT INTO log_errores_carga (origen, linea_raw, motivo)
15 SELECT archivo_origen, concat_ws(E'\t', ...),
16         'Duplicado en staging: se conserva el primer registro del lote
17         ',
18 FROM candidatos
19 WHERE rn > 1;

```

Listing 9: Paso 6 — Duplicados dentro del lote

5.7. Paso 7 — Duplicados contra el core

Identifica registros válidos que ya existen en `evento_telemetria`. Este paso debe ejecutarse **antes** del INSERT (paso 8) para evitar registrar como duplicados los registros recién insertados.

Existen dos tipos de duplicado, ambos registrados en el log:

Tipo	Descripción
Duplicado en staging	Fila repetida dentro del mismo lote TSV (paso 6).
Duplicado contra core	La tripleta ya existe en <code>evento_telemetria</code> de cargas anteriores (paso 7).

```

1 AND EXISTS (
2   SELECT 1 FROM evento_telemetria e
3   WHERE e.id_partida = s.raw_id_partida::INT
4         AND e.id_jugador = s.raw_id_jugador::INT
5         AND e.tic = s.raw_tic::INT
6 )

```

Listing 10: Paso 7 — Duplicados contra evento_telemetria

5.8. Paso 8 — Insertar registros válidos

Se ejecuta después de los pasos 6 y 7. Aunque los duplicados internos ya fueron registrados en el paso 6, `DISTINCT ON` se mantiene como mecanismo defensivo para garantizar que solo una fila por tripleta llegue al INSERT, independientemente del estado del log. `ON CONFLICT DO NOTHING` actúa como segunda barrera de seguridad.

Requiere que `schema.sql` defina: `UNIQUE (id_partida, id_jugador, tic)` en `evento_telemetria`.

```

1 INSERT INTO evento_telemetria (
2   id_partida, id_jugador, id_sector, tic,
3   pos_x, pos_y, pos_z, angulo,
4   momentum_x, momentum_y, momentum_z,
5   fov, salud, armadura, municion
6 )
7 SELECT DISTINCT ON (
8   s.raw_id_partida::INT,

```

```

9      s.raw_id_jugador::INT,
10     s.raw_tic::INT
11 )
12     s.raw_id_partida::INT,
13     s.raw_id_jugador::INT,
14     NULLIF(TRIM(s.raw_id_sector), '')::INT,
15     s.raw_tic::INT,
16     s.raw_pos_x::NUMERIC(12,4), ...
17 FROM stg_evento_telemetria s
18 JOIN jugador j ON j.id_jugador = s.raw_id_jugador::INT
19 JOIN usuario u ON u.id_usuario = j.id_usuario
20 WHERE [todas las validaciones 1-5 pasadas]
21 ORDER BY s.raw_id_partida::INT,
22          s.raw_id_jugador::INT,
23          s.raw_tic::INT,
24          s.cargado_en ASC
25 ON CONFLICT (id_partida, id_jugador, tic) DO NOTHING;

```

Listing 11: Paso 8 — INSERT final al core

6. Reporte post-carga

Al finalizar el pipeline se ejecutan estas consultas de verificación:

```

1  -- Total cargado al staging
2  SELECT COUNT(*) AS total_staging
3  FROM stg_evento_telemetria;
4
5  -- Total acumulado en core (toda la tabla, no solo el lote actual)
6  SELECT COUNT(*) AS total_core_acumulado
7  FROM evento_telemetria;
8
9  -- Errores agrupados por motivo
10 SELECT motivo, COUNT(*) AS cantidad
11 FROM log_errores_carga
12 GROUP BY motivo
13 ORDER BY cantidad DESC;
14
15 -- Ultimos 10 errores
16 SELECT id_log, origen, motivo, registrado_en
17 FROM log_errores_carga
18 ORDER BY registrado_en DESC
19 LIMIT 10;

```

Listing 12: Queries de verificación

Nota: `total_core_acumulado` refleja el total de toda la tabla `evento_telemetria`, no únicamente lo insertado en el lote actual. Si se desea medir el impacto de una carga específica, se debe registrar el conteo antes de ejecutar el pipeline y calcular la diferencia.

7. Alineación con requerimientos

Requerimiento	Cómo lo satisface el pipeline
RF11	Valida integridad durante la carga; rechaza líneas malformadas con trazabilidad completa.
RNF06	<code>log_errores_carga</code> guarda línea original y motivo de cada rechazo, incluyendo FK inválidas y duplicados.
RNF07	El pipeline completo es un script SQL versionable y reproducible; la ruta del archivo TSV debe parametrizarse o ajustarse según el entorno de ejecución.
Ética §1.1	El paso 5 impide que datos de usuarios sin consentimiento lleguen al core.